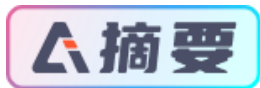


FreeRTOS互斥量 基于STM32



本文详细介绍了FreeRTOS中互斥量的基本概念、优先级继承机制及其应用场景，并通过实验演示了如何利用互斥量降低优先级翻转的风险。

文章目录

[一、互斥量基本概念](#)

[二、互斥量的优先级继承机制](#)

[三、互斥量应用场景](#)

[四、互斥量运作机制](#)

[五、互斥量函数接口讲解](#)

[1.互斥量创建函数 xSemaphoreCreateMutex\(\)](#)

[2.递归xSemaphoreCreateRecursiveMutex\(\)](#)

[3.互斥量删除函数 vSemaphoreDelete\(\)](#)

[4.互斥量获取函数 xSemaphoreTake\(\)](#)

[5.递归互斥量获取函数 xSemaphoreTakeRecursive\(\)](#)

[6.互斥量释放函数 xSemaphoreGive\(\)](#)

[7.递归互斥量释放函数 xSemaphoreGiveRecursive\(\)](#)

[六、互斥量实验](#)

[1.拟优先级翻转实验](#)

[2.互斥量实验](#)

[七、实验现象](#)

一、互斥量基本概念

互斥量又称互斥**信号量**（本质是信号量），是一种特殊的二值信号量，它和信号量不同的是，它支持互斥量所有权、递归访问以及防止优先级翻转的特性，用于实现对临界资源的独占式处理。任意时刻互斥量的状态只有两种，开锁或闭锁。当互斥量被任务持有时，该互斥量处于闭锁状态，这个任务获得互斥量的所有权。当该任务释放这个互斥量时，该互斥量处于开锁状态，任务失去该互斥量的所有权。当一个任务持有互斥量时，其他任务将不能再对该互斥量进行开锁或持有。持有该互斥量的任务也能够再次获得这个锁而不被挂起，这就是递归访问，也就是递归互斥量的特性，这个特性与一般的信号量有很大的不同，在信号量中，由于已经不存在可用的信号量，任务递归获取信号量时会发生主动挂起任务最终形成死锁。如果想要用于实现同步（任务之间或者任务与中断之间），二值信号量或许是更好的选择，虽然互斥量也可以用于任务与任务、任务与中断的同步，但是互斥量更多的是用于保护资源的互锁。用于互锁的互斥量可以充当保护资源的令牌，当一个任务希望访问某个资源时，它必须先获取令牌。当任务使用完资源后，必须还回令牌，以便其它任务可以访问该资源。是不是很熟悉，在我们的二值信号量里面也是一样的，用于保护**临界资源**，保证多任务的访问井然有序。当任务获取到信号量的时候才能开始使用被保护的资源，使用完就释放信号量，下一个任务才能获取到信号量从而可用使用被保护的资源。但是信号量会导致的另一个潜在问题，那就是任务优先级翻转（具体会在下文讲解）。

而 FreeRTOS 提供的互斥量 可以通过优先级继承算法，可以降低优先级翻转问题产生的影响，所以，用于临界资源的 保护一般建议使用互斥量。

二、互斥量的优先级继承机制

在 FreeRTOS 操作系统中为了降低优先级翻转问题利用了优先级继承算法。优先级继承算法是指，暂时提高某个占有某种资源的低优先级任务的优先级，使之与在所有等待该资源的任务中优先级最高那个任务的优先级相等，而当这个低优先级任务执行完毕释放该资源时，优先级重新回到初始设定值。因此，继承优先级的任务避免了系统资源被任何中间优先级的任务抢占。互斥量与二值信号量最大的不同是：互斥量具有优先级继承机制，而信号量没有。也就是说，某个临界资源受到一个互斥量保护，如果这个资源正在被一个低优先级任务使用，那么此时的互斥量是闭锁状态，也代表了没有任务能申请到这个互斥量，如果此时一个高优先级任务想要对这个资源进行访问，去申请这个互斥量，那么高优先级任务会因为申请不到互斥量而进入阻塞态，那么系统会将现在持有该互斥量的任务的优先级临时提升到与高优先级任务的优先级相同，这个优先级提升的过程叫做优先级继承。这个优先级继承机制确保高优先级任务进入阻塞状态的时间尽可能短，以及将已经出现的“优先级翻转”危害降低到最小。没有理解？没问题，结合过程示意图再说一遍。我们知道任务的优先级在创建的时候就已经是设置好的，高优先级的任务可以打断低优先级的任务，抢占 CPU 的使用权。但是在很多场合中，某些资源只有一个，当低优先级任务正在占用该资源的时候，即便高优先级任务也只能乖乖的等待低优先级任务使用完该资源后释放资源。这里高优先级任务无法运行而低优先级任务可以运行的现象称为“优先级翻转”。为什么说优先级翻转在操作系统中是危害很大？因为在我们一开始创造这个系统的时候，我们就已经设置好了任务的优先级了，越重要的任务优先级越高。但是发生优先级翻转，对我们操作系统是致命的危害，会导致系统的高优先级任务阻塞时间过长。

举个例子，现在有 3 个任务分别为 H 任务（High）、M 任务（Middle）、L 任务（Low），3 个任务的优先级顺序为 H 任务>M 任务>L 任务。正常运行时候 H 任务可以打断 M 任务与 L 任务，M 任务可以打断 L 任务，假设系统中有一个资源被保护了，此时该资源被 L 任务正在使用中，某一刻，H 任务需要使用该资源，但是 L 任务还没使用完，H 任务则因为申请不到资源而进入阻塞态，L 任务继续使用该资源，此时已经出现了“优先级翻转”现象，高优先级任务在等着低优先级的任务执行，如果在 L 任务执行的时候刚好 M 任务被唤醒了，由于 M 任务优先级比 L 任务优先级高，那么会打断 L 任务，抢占了 CPU 的使用权，直到 M 任务执行完，再把 CPU 使用权归还给 L 任务，L 任务继续执行，等到执行完毕之后释放该资源，H 任务此时才从阻塞态解除，使用该资源。这个过程，本来是最高优先级的 H 任务，在等待了更低优先级的 L 任务与 M 任务，其阻塞的时间是 M 任务运行时间+L 任务运行时间，这只是只有 3 个任务的系统，假如很多个这样子的任务打断最低优先级的任务，那这个系统最高优先级任务岂不是崩溃了，这个现象是绝对不允许出现的，高优先级的任务必须能及时响应。所以，没有优先级继承的情况下，使用资源保护，其危害极大。

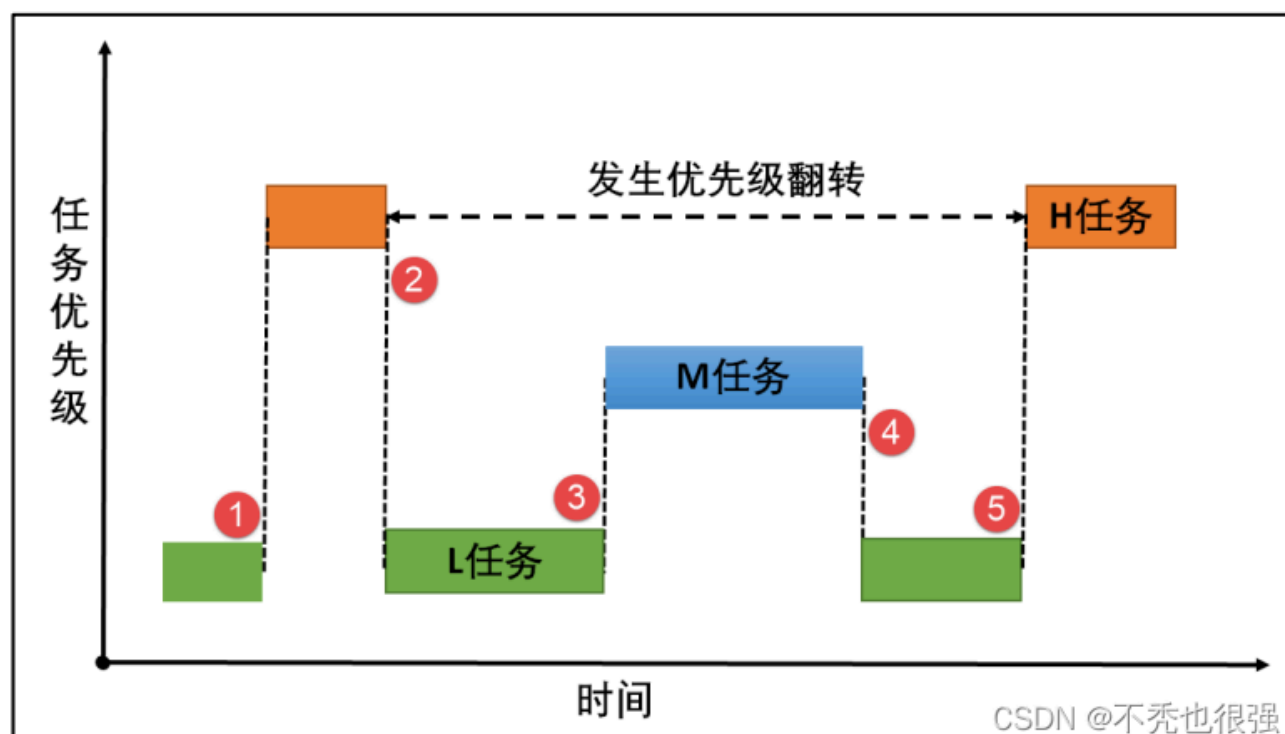


图1优先级翻转图解

图1(1) L 任务正在使用某临界资源， H 任务被唤醒，执行 H 任务。但 L 任务并未执行完毕，此时临界资源还未释放。

图1(2)：这个时刻 H 任务也要对该临界资源进行访问，但 L 任务还未释放资源， 由于保护机制， H 任务进入阻塞态， L 任务得以继续运行，此时已经发生了优先级翻转现象。

图 1(3)：某个时刻 M 任务被唤醒，由于 M 任务的优先级高于 L 任务， M 任务抢 占了 CPU 的使用权， M任务开始运行，此时 L 任务尚未执行完，临界资源还没被释放。

图 1(4)： M 任务运行结束，归还 CPU 使用权， L 任务继续运行。

图 1(5)： L任务运行结束，释放临界资源， H 任务得以对资源进行访问， H 任务开 始运行。

在这过程中， H 任务的等待时间过长，这对系统来说这是很致命的，所以这种情况不 允许出现，而互斥量就是用来降低优先级翻转的产生的危害。 假如有优先级继承呢？那么，在 H 任务申请该资源的时候，由于申请不到资源会进入阻 塞态，那么系统就会把当前正在使用资源的 L 任务的优先级临时提高到与 H 任务优先级 相同，此时 M 任务被唤醒了，因为它的优先级比 H 任务低，所以无法打断 L 任务，因为 此时 L 任务的优先级被临时提升到 H，所以当 L 任务使用完该资源了，进行释放，那么此 时 H 任务优先级最高，将接着抢占 CPU 的使用权， H 任务的阻塞时间仅仅是 L 任务的执 行时间，此时的优先级的危害降到了最低，看！这就是优先级继承的优势，

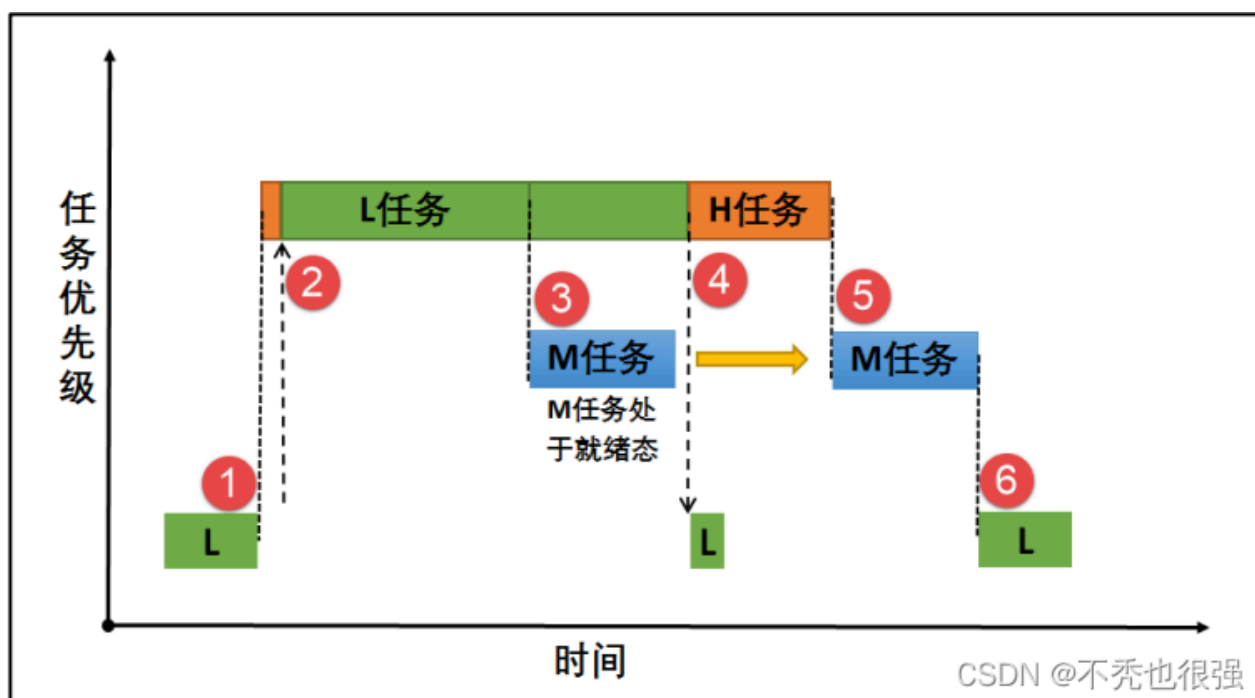


图2优先级继承

图2(1)L 任务正在使用某临界资源， L 任务正在使用某临界资源， H 任务被唤醒，执行 H 任务。但 L 任务并未执行完 毕，此时临界资源还未释放。

图 2(2)：某一时刻 H 任务也要对该资源进行访问，由于保护机制， H 任务进入阻 塞态。此时发生优先级继承，系统将 L 任务的优先级暂时提升到与 H 任务优先级相同， L 任务继续执行。

图 2(3)：在某一时刻 M 任务被唤醒，由于此时 M 任务的优先级暂时低于 L 任务， 所以 M 任务仅在就绪态，而无法获得 CPU 使用权。

图 2(4)： L任务运行完毕， H 任务获得对资源的访问权， H 任务从阻塞态变成运行 态，此时 L 任务的优先级会变回原来的 优先级。

图 2(5)：当 H 任务运行完毕， M任务得到 CPU 使用权，开始执行。

图 2(6)：系统正常运行，按照设定好的优先级运行。但是使用互斥量的时候一定需要注意：在获得互斥量后，请尽快释 放互斥量，同时需要注意的是在任务持有互斥量的这段时间，不得更改任务的优先级。FreeRTOS 的优先级继 承机制不能解决优先级反转，只能将这种情况的影响降低到最小，硬实时系统在一开始设 计时就要避免优先级反转发生。

三、互斥量应用场景

互斥量的使用比较单一，因为它是信号量的一种，并且它是以锁的形式存在。在初始化的时候，互斥量处于开锁的状态，而被任务持有的时候则立刻转为闭锁的状态。互斥量 更适合于：

可能会引起优先级翻转的情况。

递归互斥量更适用于：

任务可能会多次获取互斥量的情况下。这样可以避免同一任务多次递归持有而造成死锁的问题。

多任务环境下往往存在多个任务竞争同一临界资源的应用场景，互斥量可被用于对临界资源的保护从而实现独占式访问。另外，互斥量可以降低信号量存在的优先级翻转问题带来的影响。比如有两个任务需要对串口进行发送数据，其硬件资源只有一个，那么两个任务肯定不能同时发送啦，不然导致数据错误，那么，就可以用互斥量对串口资源进行保护，当一个任务正在使用串口的时候，另一个任务则无法使用串口，等到任务使用串口完毕之后，另外一个任务才能获得串口的使用权。另外需要注意的是互斥量不能在中断服务函数中使用，因为其特有的优先级继承机制只在任务起作用，在中断的上下文环境毫无意义。

四、互斥量运作机制

多任务环境下会存在多个任务访问同一临界资源的场景，该资源会被任务独占处理。其他任务在资源被占用的情况下不允许对该临界资源进行访问，这个时候就需要用到 FreeRTOS 的互斥量来进行资源保护，那么互斥量是怎样来避免这种冲突？用互斥量处理不同任务对临界资源的同步访问时，任务想要获得互斥量才能进行资源访问，如果一旦有任务成功获得了互斥量，则互斥量立即变为闭锁状态，此时其他任务会因为获取不到互斥量而不能访问这个资源，任务会根据用户自定义的等待时间进行等待，直到互斥量被持有的任务释放后，其他任务才能获取互斥量从而得以访问该临界资源，此时互斥量再次上锁，如此一来就可以确保每个时刻只有一个任务正在访问这个临界资源，保证了临界资源操作的安全性。

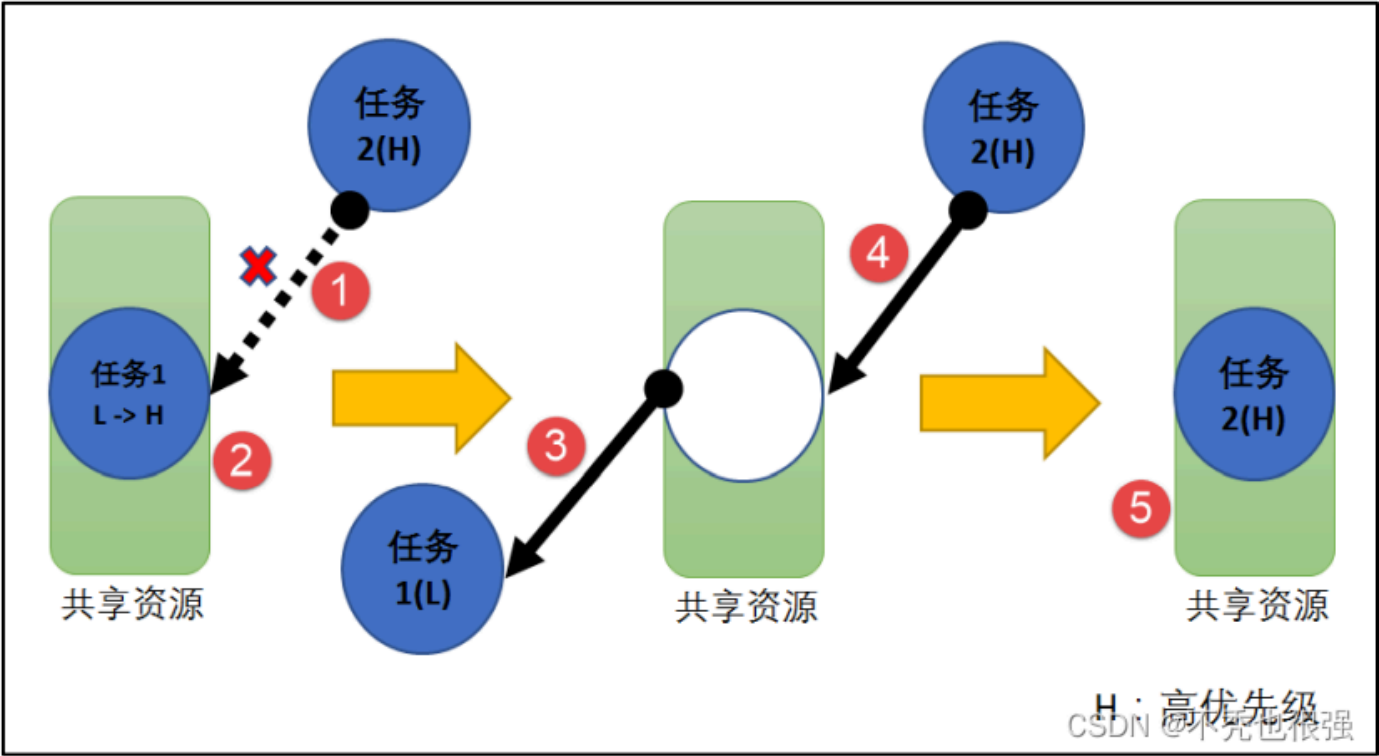


图3互斥量运作机制

五、互斥量函数接口讲解

1.互斥量创建函数 xSemaphoreCreateMutex()

xSemaphoreCreateMutex()用于创建一个互斥量，并返回一个互斥量句柄。该句柄的原型是一个 void 型的指针，在使用之前必须先由用户定义一个互斥量句柄。要想使用该函数 必须在 FreeRTOSConfig.h 中把宏 configSUPPORT_DYNAMIC_ALLOCATION 定义为 1，即开启动态内存分配，其实该宏在 FreeRTOS.h 中默认定义为 1，即所有 FreeRTOS 的对象 在创建的时候都默认使用动态内存分配方案，同时还需在 FreeRTOSConfig.h 中把 configUSE_MUTEXES 宏定义打开，表示使用互斥量。

xSemaphoreCreateMutex()函数使用实例

```
1. SemaphoreHandle_t MuxSem_Handle;
2.
3. void vATask( void * pvParameters )
4. {
5.     /* 创建一个互斥量 */
6.     MuxSem_Handle= xSemaphoreCreateMutex();
7.
8.     if (MuxSem_Handle!= NULL ) {
9.         /* 互斥量创建成功 */
10.    }
11. }
```



2.递归xSemaphoreCreateRecursiveMutex()

用于创建一个递归互斥量，不是递归的互斥量由函数 xSemaphoreCreateMutex() 或 xSemaphoreCreateMutexStatic() 创建（我们只讲解动态创建），且只能被同一个任务获取一次，如果同一个任务想再次获取则会失败。递归信号量则相反，它可以被同一个任务获取很多次，获取多少次就需要释放多少次。递归信号量与 互斥量一样，都实现了优先级继承机制，可以降低优先级反转的危害。

xSemaphoreCreateRecursiveMutex()函数说明

函数原型	#if((configSUPPORT_DYNAMIC_ALLOCATION==1) && (configUSE_RECURSIVE_MUTEXES ==1)) #define xSemaphoreCreateRecursiveMutex() xQueueCreateMutex(queueQUEUE_TYPE_RECURSIVE_MUTEX) #endif	
功能	创建一个递归互斥量。	
参数	void	无。
返回值	如果创建成功则返回一个递归互斥量句柄，用于访问创建的递归互斥量。如果创建不成功则返回 NULL。	

CSDN @不秃也很强

xSemaphoreCreateRecursiveMutex()函数使用实例

```
1. SemaphoreHandle_t xMutex;
2.
3. void vATask( void * pvParameters )
4. {
5.     /* 创建一个递归互斥量 */
6.     xMutex = xSemaphoreCreateRecursiveMutex();
7.
8.     if ( xMutex != NULL ) {
9.         /* 递归互斥量创建成功 */
10.    }
11. }
```



3.互斥量删除函数 vSemaphoreDelete()

互斥量的本质是信号量，直接调用 vSemaphoreDelete()函数进行删除即可。

4.互斥量获取函数 xSemaphoreTake()

我们知道，当互斥量处于开锁的状态，任务才能获取互斥量成功，当任务持有了某个互斥量的时候，其它任务就无法获取这个互斥量，需要等到持有互斥量的任务进行释放后，其他任务才能获取成功，任务通过互斥量获取函数来获取互斥量的所有权。任务对互斥量的所有权是独占的，任意时刻互斥量只能被一个任务持有，如果互斥量处于开锁状态，那么获取该互斥量的任务将成功获得该互斥量，并拥有互斥量的使用权；如果互斥量处于闭锁状态，获取该互斥量的任务将无法获得互斥量，任务将被挂起，在任务被挂起之前，会进行优先级继承，如果当前任务优先级比持有互斥量的任务优先级高，那么将会临时提升持有互斥量任务的优先级。互斥量的获取函数是一个宏定义，实际调用的函数就是 xQueueGenericReceive()

xSemaphoreTake()函数使用实例

```
1. static void LowPriority_Task(void* parameter)
2. {
3.     static uint32_t i;
4.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
5.     while (1)
6.     {
7.         printf("LowPriority_Task 获取互斥量\n");
8.         //获取互斥量 MuxSem, 没获取到则一直等待
9.         xReturn = xSemaphoreTake(MuxSem_Handle, /* 互斥量句柄 */
10.                                portMAX_DELAY); /* 等待时间 */
11.         if(pdTRUE == xReturn)
12.             printf("LowPriority_Task Runing\n\n");
13.
14.         for(i=0;i<2000000;i++)//模拟低优先级任务占用互斥量
15.         {
16.             taskYIELD(); //发起任务调度
17.         }
18.
19.         printf("LowPriority_Task 释放互斥量!\r\n");
20.         xReturn = xSemaphoreGive(MuxSem_Handle); //给出互斥量
21.
22.         LED1_TOGGLE;
23.
24.         vTaskDelay(1000);
25.     }
26. }
```



5.递归互斥量获取函数 xSemaphoreTakeRecursive()

xSemaphoreTakeRecursive()是一个用于获取递归互斥量的宏，与互斥量的获取函数一样，xSemaphoreTakeRecursive()也是一个宏定义，它最终使用现有的队列机制，实际执行的函数是 xQueueTakeMutexRecursive()。互斥量之前必须由 xSemaphoreCreateRecursiveMutex()这个函数创建。要注意的是该函数不能用于获取由函数 xSemaphoreCreateMutex()创建的互斥量。要想使用该函数必须在头文件 FreeRTOSConfig.h 中把宏 configUSE_RECURSIVE_MUTEXES 定义为 1。

xSemaphoreTakeRecursive()函数说明

函数原型	<pre>#if(configUSE_RECURSIVE_MUTEXES == 1) #define xSemaphoreTakeRecursive(xMutex, xBlockTime) xQueueTakeMutexRecursive((xMutex), (xBlockTime)) #endif</pre>	
功能	获取递归互斥量。	
参数	xMutex	信号量句柄。
	xBlockTime	如果不是持有互斥量的任务去获取无效的互斥量，那么任务将进行等待用户指定超时时间，单位为 tick（即系统节拍周期）。如果宏 INCLUDE_vTaskSuspend 定义为 1 且形参 xTicksToWait 设置为 portMAX_DELAY，则任务将一直阻塞在该递归互斥量上（即没有超时时间）。
返回值	获取成功则返回 pdTRUE，在超时之前没有获取成功则返回 errQUEUE_EMPTY。	

CSDN @不秃也很强

递归互斥量可以在一个任务中多次获取，当第一次获取递归互斥量时，队列结构体成员指针 pxMutexHolder 指向获取递归互斥量的任务控制块，当任务再次尝试获取这个递归互斥量时，如果任务就是拥有递归互斥量所有权的任务，那么只需要将记录获取递归次数的成员变量 u.uxRecursiveCallCount 加 1 即可，不需要再操作队列。

xSemaphoreTakeRecursive()函数的使用实例

```
1. semaphoreHandle_t xMutex = NULL;
2.
3. /* 创建信号量的任务 */
4. void vATask( void * pvParameters )
5. {
6.     /* 创建一个递归互斥量，保护共享资源 */
7.     xMutex = xSemaphoreCreateRecursiveMutex();
8. }
9.
10. /* 使用互斥量 */
11. void vAnotherTask( void * pvParameters )
12. {
13.     /* ... 做其他的事情 */
14.
15.     if ( xMutex != NULL ) {
16.         /* 尝试获取递归信号量。
17.          * 如果信号量不可用则等待 10 个 ticks */
18.         if(xSemaphoreTakeRecursive(xMutex,( TickType_t)10)==pdTRUE ) {
19.             /* 获取到递归信号量，可以访问共享资源 */
20.             /* ... 其他功能代码 */
21.
22.             /* 重复获取递归信号量 */
23.             xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 );
24.             xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 );
25.
26.             /* 释放递归信号量，获取了多少次就要释放多少次 */
27.             xSemaphoreGiveRecursive( xMutex );
28.             xSemaphoreGiveRecursive( xMutex );
29.             xSemaphoreGiveRecursive( xMutex );
30.
31.             /* 现在递归互斥量可以被其他任务获取 */
32.         } else {
33.             /* 没能成功获取互斥量，所以不能安全的访问共享资源 */
34.         }
35.     }
36. }
```

6.互斥量释放函数 xSemaphoreGive()

任务想要访问某个资源的时候，需要先获取互斥量，然后进行资源访问，在任务使用完该资源的时候，必须要及时归还互斥量，这样别的任务才能对资源进行访问。在前面的讲解中，我们知道，当互斥量有效的时候，任务才能获取互斥量，那么，是什么函数使得信号量变得有效呢？FreeRTOS 给我们提供了互斥量释放函数 xSemaphoreGive()，任务可以调用 xSemaphoreGive()函数进行释放互斥量，表示我已经用完了，别人可以申请使用，互斥量的释放函数与信号量的释放函数一致，都是调用 xSemaphoreGive()函数，但是要注意的是，互斥量的释放只能在任务中，不允许在中断中释放互斥量。使用该函数接口时，只有已持有互斥量所有权的任务才能释放它，当任务调用 xSemaphoreGive()函数时会将互斥量变为开锁状态，等待获取该互斥量的任务将被唤醒。如果任务的优先级被互斥量的优先级翻转机制临时提升，那么当互斥量被释放后，任务的优先级将恢复为原本设定的优先级

xSemaphoreGive()使用实例

```
1. SemaphoreHandle_t xSemaphore = NULL;
2. void vATask( void * pvParameters )
3. {
4.     /* 创建一个互斥量用于保护共享资源 */
5.     xSemaphore = xSemaphoreCreateMutex();
6.
7.     if ( xSemaphore != NULL ) {
8.         if ( xSemaphoreGive( xSemaphore ) != pdTRUE ) {
9.             /* 如果要释放一个互斥量，必须先有第一次的获取*/
10.        }
11.
12.        /* 获取互斥量，不等待 */
13.        if ( xSemaphoreTake( xSemaphore, ( TickType_t ) 0 ) ) {
14.            /* 获取到互斥量，可以访问共享资源 */
15.
16.            /* ... 访问共享资源代码 */
17.
18.            /* 共享资源访问完毕，释放互斥量 */
19.            if ( xSemaphoreGive( xSemaphore ) != pdTRUE ) {
20.                /* 互斥量释放失败，这可不是我们希望的 */
21.            }
22.        }
23.    }
24. }
```

7.递归互斥量释放函数 xSemaphoreGiveRecursive()

xSemaphoreGiveRecursive()是一个用于释放递归互斥量的宏。要想使用该函数必须在头文件 FreeRTOSConfig.h 把宏 configUSE_RECURSIVE_MUTEXES 定义为 1。互斥量和递归互斥量的最大区别在于一个递归互斥量可以被已经获取这个递归互斥量的任务重复获取，而不会形成死锁。这个递归调用功能是通过队列结构体成员 u.xRecursiveCallCount 实现的，这个变量用于存储递归调用的次数，每次获取递归互斥量后，这个变量加 1，在释放递归互斥量后，这个变量减 1。只有这个变量减到 0，即释放和获取的次数相等时，互斥量才能变成有效状态，然后才允许使用 xQueueGenericSend()函数释放一个递归互斥量。

```
1. SemaphoreHandle_t xMutex = NULL;
2. void vATask( void * pvParameters )
3. {
4.     /* 创建一个递归互斥量用于保护共享资源 */
5.     xMutex = xSemaphoreCreateRecursiveMutex();
6. }
7. void vAnotherTask( void * pvParameters )
8. {
9.     /* 其他功能代码 */
10.
11.     if ( xMutex != NULL ) {
12.         /* 尝试获取递归互斥量
13.          * 如果不可用则等待 10 个 ticks */
14.         if(xSemaphoreTakeRecursive(xMutex,( TickType_t ) 10 )== pdTRUE) {
15.             /* 获取到递归信号量，可以访问共享资源 */
16.             /* ... 其他功能代码 */
17.
18.             /* 重复获取递归互斥量 */
19.             xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 );
20.             xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 );
21.
22.             /* 释放递归互斥量，获取了多少次就要释放多少次 */
23.             xSemaphoreGiveRecursive( xMutex );
24.             xSemaphoreGiveRecursive( xMutex );
25.             xSemaphoreGiveRecursive( xMutex );
26. }
```



```

26.
27.         /* 现在递归互斥量可以被其他任务获取 */
28.     } else {
29.         /* 没能成功获取互斥量，所以不能安全的访问共享资源 */
30.     }
31. }
32. }

```



六、互斥量实验

1.拟优先级翻转实验

模拟优先级翻转实验是在 FreeRTOS 中创建了三个任务与一个二值信号量，任务分别是高优先级任务，中优先级任务，低优先级任务，用于模拟产生优先级翻转。低优先级任务在获取信号量的时候，被中优先级打断，中优先级的任务执行时间较长，因为低优先级还未释放信号量，那么高优先级任务就无法取得信号量继续运行，此时就发生了优先级翻转，任务在运行中，使用串口打印出相关信息

```

1.  /* FreeRTOS头文件 */
2.  #include "FreeRTOS.h"
3.  #include "task.h"
4.  #include "queue.h"
5.  #include "semphr.h"
6.  /* 开发板硬件bsp头文件 */
7.  #include "bsp_led.h"
8.  #include "bsp_usart.h"
9.  #include "bsp_key.h"
10. /***** 任务句柄 *****/
11. /*
12.  * 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
13.  * 以后我们要想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
14.  * 这个句柄可以为NULL。
15.  */
16. static TaskHandle_t AppTaskCreate_Handle = NULL; /* 创建任务句柄 */
17. static TaskHandle_t LowPriority_Task_Handle = NULL; /* LowPriority_Task任务句柄 */
18. static TaskHandle_t MidPriority_Task_Handle = NULL; /* MidPriority_Task任务句柄 */
19. static TaskHandle_t HighPriority_Task_Handle = NULL; /* HighPriority_Task任务句柄 */
20.
21. /***** 内核对象句柄 *****/
22. /*
23.  * 信号量，消息队列，事件标志组，软件定时器这些都属于内核的对象，要想使用这些内核
24.  * 对象，必须先创建，创建成功之后会返回一个相应的句柄。实际上就是一个指针，后续我
25.  * 们就可以通过这个句柄操作这些内核对象。
26.  *
27.  * 内核对象说白了就是一种全局的数据结构，通过这些数据结构我们可以实现任务间的通信，
28.  * 任务间的事件同步等各种功能。至于这些功能的实现我们是通过调用这些内核对象的函数
29.  * 来完成的
30.  *
31.  */
32. SemaphoreHandle_t BinarySem_Handle = NULL;
33.
34. /***** 全局变量声明 *****/
35. /*
36.  * 当我们在写应用程序的时候，可能需要用到一些全局变量。
37.  */
38.
39.
40. /***** 宏定义 *****/
41. /*
42.  * 当我们在写应用程序的时候，可能需要用到一些宏定义。
43.  */
44.
45.
46. /*
47.  *****/
48.  * 函数声明
49.  *****/
50. /*
51. static void AppTaskCreate(void); /* 用于创建任务 */
52.
53. static void LowPriority_Task(void* pvParameters); /* LowPriority_Task任务实现 */
54. static void MidPriority_Task(void* pvParameters); /* MidPriority_Task任务实现 */
55. static void HighPriority_Task(void* pvParameters); /* MidPriority_Task任务实现 */
56.
57. static void BSP_Init(void); /* 用于初始化板载相关资源 */
58.

```

```

59. /*****
60.  * @brief 主函数
61.  * @param 无
62.  * @retval 无
63.  * @note 第一步：开发板硬件初始化
64.          第二步：创建APP应用任务
65.          第三步：启动FreeRTOS，开始多任务调度
66.  *****/
67. int main(void)
68. {
69.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
70.
71.     /* 开发板硬件初始化 */
72.     BSP_Init();
73.     printf("这是一个FreeRTOS优先级翻转实验！\n");
74.     /* 创建AppTaskCreate任务 */
75.     xReturn = xTaskCreate((TaskFunction_t)AppTaskCreate, /* 任务入口函数 */
76.                           (const char*) "AppTaskCreate", /* 任务名字 */
77.                           (uint16_t) 512, /* 任务栈大小 */
78.                           (void*) NULL, /* 任务入口函数参数 */
79.                           (UBaseType_t) 1, /* 任务的优先级 */
80.                           (TaskHandle_t*) &AppTaskCreate_Handle); /* 任务控制块指针 */
81.     /* 启动任务调度 */
82.     if (pdPASS == xReturn)
83.         vTaskStartScheduler(); /* 启动任务，开启调度 */
84.     else
85.         return -1;
86.
87.     while(1); /* 正常不会执行到这里 */
88. }
89.
90. /*****
91.  * @ 函数名 : AppTaskCreate
92.  * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
93.  * @ 参数 : 无
94.  * @ 返回值 : 无
95.  *****/
96. static void AppTaskCreate(void)
97. {
98.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
99.
100.    taskENTER_CRITICAL(); /* 进入临界区 */
101.
102.    /* 创建Test_Queue */
103.    BinarySem_Handle = xSemaphoreCreateBinary();
104.    if (NULL != BinarySem_Handle)
105.        printf("BinarySem_Handle二值信号量创建成功!\r\n");
106.
107.    xReturn = xSemaphoreGive(BinarySem_Handle); /* 给出二值信号量 */
108.    // if (xReturn == pdTRUE)
109.    //     printf("释放信号量!\r\n");
110.
111.    /* 创建LowPriority_Task任务 */
112.    xReturn = xTaskCreate((TaskFunction_t)LowPriority_Task, /* 任务入口函数 */
113.                          (const char*) "LowPriority_Task", /* 任务名字 */
114.                          (uint16_t) 512, /* 任务栈大小 */
115.                          (void*) NULL, /* 任务入口函数参数 */
116.                          (UBaseType_t) 2, /* 任务的优先级 */
117.                          (TaskHandle_t*) &LowPriority_Task_Handle); /* 任务控制块指针 */
118.
119.    if (pdPASS == xReturn)
120.        printf("创建LowPriority_Task任务成功!\r\n");
121.
122.    /* 创建MidPriority_Task任务 */
123.    xReturn = xTaskCreate((TaskFunction_t)MidPriority_Task, /* 任务入口函数 */
124.                          (const char*) "MidPriority_Task", /* 任务名字 */
125.                          (uint16_t) 512, /* 任务栈大小 */
126.                          (void*) NULL, /* 任务入口函数参数 */
127.                          (UBaseType_t) 3, /* 任务的优先级 */
128.                          (TaskHandle_t*) &MidPriority_Task_Handle); /* 任务控制块指针 */
129.
130.    if (pdPASS == xReturn)
131.        printf("创建MidPriority_Task任务成功!\n");
132.
133.    /* 创建HighPriority_Task任务 */
134.    xReturn = xTaskCreate((TaskFunction_t)HighPriority_Task, /* 任务入口函数 */
135.                          (const char*) "HighPriority_Task", /* 任务名字 */
136.                          (uint16_t) 512, /* 任务栈大小 */
137.                          (void*) NULL, /* 任务入口函数参数 */
138.                          (UBaseType_t) 4, /* 任务的优先级 */
139.                          (TaskHandle_t*) &HighPriority_Task_Handle); /* 任务控制块指针 */
140.
141.    if (pdPASS == xReturn)
142.        printf("创建HighPriority_Task任务成功!\n\n");
143.

```

```

142.     vTaskDelete(AppTaskCreate_Handle); //删除AppTaskCreate任务
143.
144.     taskEXIT_CRITICAL();           //退出临界区
145. }
146.
147.
148.
149. /*****
150.  * @ 函数名 : LowPriority_Task
151.  * @ 功能说明: LowPriority_Task任务主体
152.  * @ 参数 :
153.  * @ 返回值 : 无
154.  *****/
155. static void LowPriority_Task(void* parameter)
156. {
157.     static uint32_t i;
158.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
159.     while (1)
160.     {
161.         printf("LowPriority_Task 获取信号量\n");
162.         //获取二值信号量 xSemaphore, 没获取到则一直等待
163.         xReturn = xSemaphoreTake(BinarySem_Handle, /* 二值信号量句柄 */
164.                                 portMAX_DELAY); /* 等待时间 */
165.         if( xReturn == pdTRUE )
166.             printf("LowPriority_Task Runing\n\n");
167.
168.         for(i=0;i<2000000;i++)//模拟低优先级任务占用信号量
169.         {
170.             taskYIELD(); //发起任务调度
171.         }
172.
173.         printf("LowPriority_Task 释放信号量!\r\n");
174.         xReturn = xSemaphoreGive( BinarySem_Handle ); //给出二值信号量
175.         // if( xReturn == pdTRUE )
176.         // ; /* 什么都不做 */
177.
178.         LED1_TOGGLE;
179.
180.         vTaskDelay(500);
181.     }
182. }
183.
184. /*****
185.  * @ 函数名 : MidPriority_Task
186.  * @ 功能说明: MidPriority_Task任务主体
187.  * @ 参数 :
188.  * @ 返回值 : 无
189.  *****/
190. static void MidPriority_Task(void* parameter)
191. {
192.     while (1)
193.     {
194.         printf("MidPriority_Task Runing\n");
195.         vTaskDelay(500);
196.     }
197. }
198.
199. /*****
200.  * @ 函数名 : HighPriority_Task
201.  * @ 功能说明: HighPriority_Task 任务主体
202.  * @ 参数 :
203.  * @ 返回值 : 无
204.  *****/
205. static void HighPriority_Task(void* parameter)
206. {
207.     BaseType_t xReturn = pdTRUE; /* 定义一个创建信息返回值，默认为pdPASS */
208.     while (1)
209.     {
210.         printf("HighPriority_Task 获取信号量\n");
211.         //获取二值信号量 xSemaphore, 没获取到则一直等待
212.         xReturn = xSemaphoreTake(BinarySem_Handle, /* 二值信号量句柄 */
213.                                 portMAX_DELAY); /* 等待时间 */
214.         if(pdTRUE == xReturn)
215.             printf("HighPriority_Task Runing\n");
216.             LED1_TOGGLE;
217.         xReturn = xSemaphoreGive( BinarySem_Handle ); //互斥量释放函数 给出二值信号量
218.         // if( xReturn == pdTRUE )
219.         // //printf("HighPriority_Task 释放信号量!\r\n");
220.
221.         vTaskDelay(500);
222.     }
223. }
224.

```

```

225.
226. /*****
227.  * @ 函数名 : BSP_Init
228.  * @ 功能说明: 板级外设初始化, 所有板子上的初始化均可放在这个函数里面
229.  * @ 参数 :
230.  * @ 返回值 : 无
231.  *****/
232. static void BSP_Init(void)
233. {
234.     /*
235.     * STM32中断优先级分组为4, 即4bit都用来表示抢占优先级, 范围为: 0~15
236.     * 优先级分组只需要分组一次即可, 以后如果有其他的任务需要用到中断,
237.     * 都统一用这个优先级分组, 千万不要再分组, 切忌。
238.     */
239.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
240.
241.     /* LED 初始化 */
242.     LED_GPIO_Config();
243.
244.     /* 串口初始化 */
245.     USART_Config();
246.
247.     /* 按键初始化 */
248.     Key_GPIO_Config();
249. }
250. }
251.
252. /*****END OF FILE*****/

```



2.互斥量实验

互斥量实验是基于优先级翻转实验进行修改的, 目的是为了测试互斥量的优先级继承 机制是否有效

```

1.
2. /* FreeRTOS头文件 */
3. #include "FreeRTOS.h"
4. #include "task.h"
5. #include "queue.h"
6. #include "semphr.h"
7. /* 开发板硬件bsp头文件 */
8. #include "bsp_led.h"
9. #include "bsp_usart.h"
10. #include "bsp_key.h"
11. /***** 任务句柄 *****/
12. /*
13.  * 任务句柄是一个指针, 用于指向一个任务, 当任务创建好之后, 它就具有了一个任务句柄
14.  * 以后我们要想操作这个任务都需要通过这个任务句柄, 如果是自身的任务操作自己, 那么
15.  * 这个句柄可以为NULL。
16.  */
17. static TaskHandle_t AppTaskCreate_Handle = NULL; /* 创建任务句柄 */
18. static TaskHandle_t LowPriority_Task_Handle = NULL; /* LowPriority_Task任务句柄 */
19. static TaskHandle_t MidPriority_Task_Handle = NULL; /* MidPriority_Task任务句柄 */
20. static TaskHandle_t HighPriority_Task_Handle = NULL; /* HighPriority_Task任务句柄 */
21. /***** 内核对象句柄 *****/
22. /*
23.  * 信号量, 消息队列, 事件标志组, 软件定时器这些都属于内核的对象, 要想使用这些内核
24.  * 对象, 必须先创建, 创建成功之后会返回一个相应的句柄。实际上就是一个指针, 后续我
25.  * 们就可以通过这个句柄操作这些内核对象。
26.  *
27.  * 内核对象说白了就是一种全局的数据结构, 通过这些数据结构我们可以实现任务间的通信,
28.  * 任务间的事件同步等各种功能。至于这些功能的实现我们是通过调用这些内核对象的函数
29.  * 来完成的
30.  */
31. /*
32. SemaphoreHandle_t MuxSem_Handle =NULL;
33.
34. /***** 全局变量声明 *****/
35. /*
36.  * 当我们在写应用程序的时候, 可能需要用到一些全局变量。
37.  */
38.
39.
40. /***** 宏定义 *****/
41. /*
42.  * 当我们在写应用程序的时候, 可能需要用到一些宏定义。
43.  */
44.
45.
46. /*

```

```

47. ****
48. * 函数声明
49. ****
50. */
51. static void AppTaskCreate(void);/* 用于创建任务 */
52.
53. static void LowPriority_Task(void* pvParameters);/* LowPriority_Task任务实现 */
54. static void MidPriority_Task(void* pvParameters);/* MidPriority_Task任务实现 */
55. static void HighPriority_Task(void* pvParameters);/* MidPriority_Task任务实现 */
56.
57. static void BSP_Init(void);/* 用于初始化板载相关资源 */
58.
59. /*****
60.  * @brief 主函数
61.  * @param 无
62.  * @retval 无
63.  * @note 第一步：开发板硬件初始化
64.          第二步：创建APP应用任务
65.          第三步：启动FreeRTOS，开始多任务调度
66.  *****/
67. int main(void)
68. {
69.     BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为pdPASS */
70.
71.     /* 开发板硬件初始化 */
72.     BSP_Init();
73.     printf("这是一个FreeRTOS互斥量实验！\n");
74.     /* 创建AppTaskCreate任务 */
75.     xReturn = xTaskCreate((TaskFunction_t )AppTaskCreate, /* 任务入口函数 */
76.                          (const char* )"AppTaskCreate",/* 任务名字 */
77.                          (uint16_t )512, /* 任务栈大小 */
78.                          (void* )NULL,/* 任务入口函数参数 */
79.                          (UBaseType_t )1, /* 任务的优先级 */
80.                          (TaskHandle_t* )&AppTaskCreate_Handle);/* 任务控制块指针 */
81.     /* 启动任务调度 */
82.     if(pdPASS == xReturn)
83.         vTaskStartScheduler(); /* 启动任务，开启调度 */
84.     else
85.         return -1;
86.
87.     while(1); /* 正常不会执行到这里 */
88. }
89.
90.
91. /*****
92.  * @ 函数名 : AppTaskCreate
93.  * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
94.  * @ 参数 : 无
95.  * @ 返回值 : 无
96.  *****/
97. static void AppTaskCreate(void)
98. {
99.     BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为pdPASS */
100.
101.     taskENTER_CRITICAL(); /*进入临界区
102.
103.     /* 创建MuxSem */
104.     MuxSem_Handle = xSemaphoreCreateMutex();
105.     if(NULL != MuxSem_Handle)
106.         printf("MuxSem_Handle互斥量创建成功!\r\n");
107.
108.     xReturn = xSemaphoreGive( MuxSem_Handle );//给出互斥量
109.     // if( xReturn == pdTRUE )
110.     //     printf("释放信号量!\r\n");
111.
112.     /* 创建LowPriority_Task任务 */
113.     xReturn = xTaskCreate((TaskFunction_t )LowPriority_Task, /* 任务入口函数 */
114.                          (const char* )"LowPriority_Task",/* 任务名字 */
115.                          (uint16_t )512, /* 任务栈大小 */
116.                          (void* )NULL, /* 任务入口函数参数 */
117.                          (UBaseType_t )2, /* 任务的优先级 */
118.                          (TaskHandle_t* )&LowPriority_Task_Handle);/* 任务控制块指针 */
119.     if(pdPASS == xReturn)
120.         printf("创建LowPriority_Task任务成功!\r\n");
121.
122.     /* 创建MidPriority_Task任务 */
123.     xReturn = xTaskCreate((TaskFunction_t )MidPriority_Task, /* 任务入口函数 */
124.                          (const char* )"MidPriority_Task",/* 任务名字 */
125.                          (uint16_t )512, /* 任务栈大小 */
126.                          (void* )NULL,/* 任务入口函数参数 */
127.                          (UBaseType_t )3, /* 任务的优先级 */
128.                          (TaskHandle_t* )&MidPriority_Task_Handle);/* 任务控制块指针 */
129.     if(pdPASS == xReturn)

```

```

130.     printf("创建MidPriority_Task任务成功!\n");
131.
132. /* 创建HighPriority_Task任务 */
133. xReturn = xTaskCreate((TaskFunction_t )HighPriority_Task, /* 任务入口函数 */
134.                      (const char* )"HighPriority_Task",/* 任务名字 */
135.                      (uint16_t )512, /* 任务栈大小 */
136.                      (void* )NULL,/* 任务入口函数参数 */
137.                      (UBaseType_t )4, /* 任务的优先级 */
138.                      (TaskHandle_t* )&HighPriority_Task_Handle);/* 任务控制块指针 */
139. if(pdPASS == xReturn)
140.     printf("创建HighPriority_Task任务成功!\n\n");
141.
142. vTaskDelete(AppTaskCreate_Handle); //删除AppTaskCreate任务
143.
144. taskEXIT_CRITICAL(); //退出临界区
145. }
146.
147.
148.
149. /*****
150.  * @ 函数名 : LowPriority_Task
151.  * @ 功能说明: LowPriority_Task任务主体
152.  * @ 参数 :
153.  * @ 返回值 : 无
154.  *****/
155. static void LowPriority_Task(void* parameter)
156. {
157.     static uint32_t i;
158.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
159.     while (1)
160.     {
161.         printf("LowPriority_Task 获取互斥量\n");
162.         //获取互斥量 MuxSem,没获取到则一直等待
163.         xReturn = xSemaphoreTake(MuxSem_Handle, /* 互斥量句柄 */
164.                                  portMAX_DELAY); /* 等待时间 */
165.         if(pdTRUE == xReturn)
166.             printf("LowPriority_Task Runing\n\n");
167.
168.         for(i=0;i<2000000;i++)//模拟低优先级任务占用互斥量
169.         {
170.             taskYIELD(); //发起任务调度
171.         }
172.
173.         printf("LowPriority_Task 释放互斥量!\r\n");
174.         xReturn = xSemaphoreGive( MuxSem_Handle );//给出互斥量
175.
176.         LED1_TOGGLE;
177.
178.         vTaskDelay(1000);
179.     }
180. }
181.
182. /*****
183.  * @ 函数名 : MidPriority_Task
184.  * @ 功能说明: MidPriority_Task任务主体
185.  * @ 参数 :
186.  * @ 返回值 : 无
187.  *****/
188. static void MidPriority_Task(void* parameter)
189. {
190.     while (1)
191.     {
192.         printf("MidPriority_Task Runing\n");
193.         vTaskDelay(1000);
194.     }
195. }
196.
197. /*****
198.  * @ 函数名 : HighPriority_Task
199.  * @ 功能说明: HighPriority_Task 任务主体
200.  * @ 参数 :
201.  * @ 返回值 : 无
202.  *****/
203. static void HighPriority_Task(void* parameter)
204. {
205.     BaseType_t xReturn = pdTRUE; /* 定义一个创建信息返回值，默认为pdPASS */
206.     while (1)
207.     {
208.         printf("HighPriority_Task 获取互斥量\n");
209.         //获取互斥量 MuxSem,没获取到则一直等待
210.         xReturn = xSemaphoreTake(MuxSem_Handle, /* 互斥量句柄 */
211.                                  portMAX_DELAY); /* 等待时间 */
212.         if(pdTRUE == xReturn)

```



```

213.     printf("HighPriority_Task Runing\n");
214.         LED1_TOGGLE;
215.
216.     printf("HighPriority_Task 释放互斥量!\r\n");
217.     xReturn = xSemaphoreGive( MuxSem_Handle );//给出互斥量
218.
219.
220.     vTaskDelay(1000);
221. }
222. }
223.
224.
225. /*****
226.  * @ 函数名 : BSP_Init
227.  * @ 功能说明: 板级外设初始化, 所有板子上的初始化均可放在这个函数里面
228.  * @ 参数 :
229.  * @ 返回值 : 无
230.  *****/
231. static void BSP_Init(void)
232. {
233.     /*
234.     * STM32中断优先级分组为4, 即4bit都用来表示抢占优先级, 范围为: 0~15
235.     * 优先级分组只需要分组一次即可, 以后如果有其他的任务需要用到中断,
236.     * 都统一用这个优先级分组, 千万不要再分组, 切忌。
237.     */
238.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
239.
240.     /* LED 初始化 */
241.     LED_GPIO_Config();
242.
243.     /* 串口初始化 */
244.     USART_Config();
245.
246.     /* 按键初始化 */
247.     Key_GPIO_Config();
248. }
249. }
250.
251. /*****END OF FILE*****/

```



七、实验现象

优先级反转

串口配置

端口: COM13 US1

波特率: 115200

数据位: 8

停止位: 1

校验位: 无

操作: ● 关闭串口

创建MidPriority_Task任务成功!

创建HighPriority_Task任务成功!

HighPriority_Task 获取互斥量

HighPriority_Task Runing

HighPriority_Task 释放互斥量!

MidPriority_Task Runing

LowPriority_Task 获取互斥量

LowPriority_Task Runing

HighPriority_Task 获取互斥量

LowPriority_Task 释放互斥量!

HighPriority_Task Runing

HighPriority_Task 释放互斥量!

MidPriority_Task Runing

HighPriority_Task 获取互斥量

HighPriority_Task Runing

HighPriority_Task 释放互斥量!

MidPriority_Task Runing

LowPriority_Task 获取互斥量

LowPriority_Task Runing

HighPriority_Task 获取互斥量

LowPriority_Task 释放互斥量!

HighPriority_Task Runing

HighPriority_Task 释放互斥量!

接收区设置

☒ ASCII ☐ HEX

CSDN @不秃也很强

互斥量

串口配置

端口COM13 USI

波特率115200

数据位8

停止位1

校验位无

操作

关闭串口

接收区设置

ASCII

HEX

停止显示

日志模式

清空接收区

保存到文件

HighPriority_Task Runing

HighPriority_Task 释放互斥量!

MidPriority_Task Runing

LowPriority_Task 获取互斥量

LowPriority_Task Runing

HighPriority_Task 获取互斥量

LowPriority_Task 释放互斥量!

HighPriority_Task Runing

HighPriority_Task 释放互斥量!

MidPriority_Task Runing

HighPriority_Task 获取互斥量

HighPriority_Task Runing

HighPriority_Task 释放互斥量!

MidPriority_Task Runing

LowPriority_Task 获取互斥量

LowPriority_Task Runing

HighPriority_Task 获取互斥量

LowPriority_Task 释放互斥量!

HighPriority_Task Runing

HighPriority_Task 释放互斥量!

MidPriority_Task Runing

HighPriority_Task 获取互斥量

HighPriority_Task Runing

HighPriority_Task 释放互斥量!

MidPriority_Task Runing

LowPriority_Task 获取互斥量

LowPriority_Task Runing

CSDN @不秃也很强